SET 2 | 45 Q

Sample Questions

Full-Length Set 2

Databricks Certified Data Engineer Associate (exam version May 4, 2026)

- 45 new scenario-based questions (no overlap with Set 1)
- Exam-weighted: D1 3 | D2 9 | D3 10 | D4 7 | D5 4 | D6 5 | D7 7
- Objective line quoted verbatim from the exam outline
- Answer key with explanations, distractor analysis, and doc URLs
- Claims checked against docs.databricks.com (Sep 2026)

Study-only material. Not affiliated with Databricks. Real exam items are not disclosed.

How to Use This Set

Each question copies the structure of the official exam guide samples: an **Objective** line quoted from the exam outline, a short scenario with concrete constraints, a one-sentence question, and four options (A to D) with exactly one correct answer. Answers and explanations are collected at the end, as in the exam guide, so you can attempt the questions without seeing them.

Section	Domain	Exam weight	Questions	Range
Section 1	Databricks Intelligence Platform	6%	3	Q1-Q3
Section 2	Data Ingestion and Loading	21%	9	Q4-Q12
Section 3	Data Transformation and Modeling	22%	10	Q13-Q22
Section 4	Working with Lakeflow Jobs	16%	7	Q23-Q29
Section 5	Implementing CI/CD	10%	4	Q30-Q33
Section 6	Troubleshooting, Monitoring, and Optimization	10%	5	Q34-Q38
Section 7	Governance and Security	15%	7	Q39-Q45

EXAM FOCUS Pacing: 45 scored questions in 90 minutes is about **2 minutes per question**. Time yourself on this set at the same pace.

TIP Pattern from the official samples: the correct option is usually the **Databricks-native, managed, or automatic** feature. Distractors tend to be (1) workable but sub-optimal manual fixes, (2) legacy or anti-pattern approaches, (3) swapped details such as formats, defaults, or limits, and (4) a real product applied to the wrong use case.

NOTE Product names follow the current exam guide: Lakeflow Jobs, Lakeflow Connect, Lakeflow Spark Declarative Pipelines, and Declarative Automation Bundles (formerly Databricks Asset Bundles).

SECTION 1 Databricks Intelligence Platform

Exam weight 6%

Question 1

Objective: *Understand the core components of the Databricks Data Intelligence Platform, such as its architecture, Delta Lake, and Unity Catalog.*

A research team needs to store thousands of PDF contracts and scanned images alongside their Delta tables. Access to the files must be governed by the same Unity Catalog permissions model, and the files must be readable from SQL, Python, and Spark by a path.

Which Unity Catalog object should the data engineer use?

- **A.** A managed Delta table with a BINARY column for every file
- **B.** A Unity Catalog volume, accessed through `/Volumes/<catalog>/<schema>/<volume>/...`
- **C.** A view that references the cloud storage path directly
- **D.** A DBFS root folder with workspace-level ACLs

Question 2

Objective: *Understand the core components of the Databricks Data Intelligence Platform, such as its architecture, Delta Lake, and Unity Catalog.*

An architect explains to a new team member why two jobs can write to the same Delta table at the same time without corrupting it, and why readers always see a consistent snapshot.

Which component of Delta Lake provides these guarantees?

- **A.** The Parquet file footer statistics
- **B.** The Unity Catalog metastore
- **C.** The file-based transaction log stored with the table's data files
- **D.** The Spark driver's in-memory cache of table versions

Question 3

Objective: *Understand Databricks Data Intelligence Platform's compute services, including their characteristics, limitations, and cost models, and select the most suitable option for each workload use case.*

Hundreds of analysts run ad hoc SQL and BI dashboards against gold tables. Demand spikes unpredictably during business hours. The platform team wants minimal management, fast scaling as queries queue up, and no paying for idle capacity.

Which compute option is most suitable?

- **A.** A serverless SQL warehouse
- **B.** A classic SQL warehouse with max clusters set to 1
- **C.** An all-purpose cluster in dedicated (single user) access mode
- **D.** Serverless compute for jobs

SECTION 2 Data Ingestion and Loading Exam weight 21%

Question 4

Objective: *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity Catalog-governed tables.*

An Auto Loader stream infers the schema of CSV files. Occasionally a value in the numeric `amount` column arrives as text (for example `"12,50"`) and cannot be parsed. The team wants the stream to keep running and to keep those raw values for later investigation.

Where does Auto Loader place the values that cannot be parsed by default?

- **A.** They are silently converted to 0
- **B.** The stream fails immediately with a type mismatch exception
- **C.** The entire file is moved to a `_bad_files` directory
- **D.** In the `_rescued_data` column that Auto Loader adds when it infers the schema

Question 5

Objective: *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity Catalog-governed tables.*

An Auto Loader pipeline on Databricks Runtime 16.4 LTS reads from a landing folder that keeps growing, which increases storage and file discovery cost. After ingestion, processed files should be archived automatically to a Unity Catalog volume.

Which configuration should the data engineer add?

- **A.** `cloudFiles.cleanSource = MOVE` with `cloudFiles.cleanSource.moveDestination` set to a volume path
- **B.** `cloudFiles.schemaEvolutionMode = none`
- **C.** `cloudFiles.useIncrementalListing = true`
- **D.** A daily `VACUUM` on the target Delta table

Question 6

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A team uses Auto Loader but only needs the data refreshed every 4 hours. They want to keep Auto Loader's exactly-once file tracking while paying for compute only while files are processed. Each run should process every new file and then stop.

Which trigger should the streaming write use when it is scheduled with Lakeflow Jobs?

- **A.** The default trigger with no trigger specified
- **B.** `.trigger(processingTime="4 hours")` on a continuously running cluster
- **C.** `.trigger(availableNow=True)`
- **D.** `.trigger(once=True)`

Question 7

Objective: *Use the COPY INTO command to incrementally load files from cloud object storage (ADLS/S3/GCS) into Unity-Catalog-governed tables.*

A SQL-only team wants to load JSON files whose schema is not known in advance. They created the target table with `CREATE TABLE IF NOT EXISTS main.bronze.events;` (no columns) and now want COPY INTO to infer and set the schema on the first load.

Which COPY INTO configuration is required?

- **A.** `FORMAT_OPTIONS ('inferSchema' = 'true')` only
- **B.** `COPY_OPTIONS ('force' = 'true')` only
- **C.** `FORMAT_OPTIONS ('mergeSchema' = 'true')` and `COPY_OPTIONS ('mergeSchema' = 'true')`
- **D.** COPY INTO cannot load into a table without a schema; the columns must be declared first

---

Question 8

Objective: *Configure Lakeflow Connect to reliably ingest data from diverse enterprise sources into Unity Catalog-governed tables.*

A company wants to replicate tables from an on-premises SQL Server database into Unity Catalog, including updates and deletes, and keep SCD Type 2 history. They choose the Lakeflow Connect SQL Server connector.

Which statement correctly describes the connector's architecture?

- **A.** A single serverless pipeline queries every table in full on each run
- **B.** An ingestion gateway extracts change data and stages it in a Unity Catalog volume, and an ingestion pipeline applies the staged changes to Delta tables
- **C.** Auto Loader reads the SQL Server transaction log files from cloud storage
- **D.** A JDBC notebook task must be scheduled for each table

---

Question 9

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A business analyst has a 40 MB CSV exported from a spreadsheet and needs it as a Unity Catalog table for a one-time analysis. The analyst has no cloud storage access and does not write code.

What is the most appropriate way to load the file?

- **A.** Configure a Lakeflow Connect managed connector
- **B.** Write an Auto Loader stream with a checkpoint location
- **C.** Ask an admin to create an external location for the analyst's laptop
- **D.** Use the workspace UI to upload the file and create a Unity Catalog managed table from it

---

Question 10

Objective: *Use JDBC/ODBC or REST clients in notebooks to land data into cloud storage or directly into Unity Catalog-governed tables, usually orchestrated and scheduled with Lakeflow Jobs.*

A notebook pulls a 500-million-row table from PostgreSQL over JDBC into a bronze table. The read runs in a single task and takes hours. The table has an evenly distributed integer primary key `id`, and the credentials are currently hard-coded in the URL.

Which change speeds up the read and follows Databricks security guidance?

- **A.** Set `partitionColumn="id"`, `lowerBound`, `upperBound`, and `numPartitions`, and read the credentials with `dbutils.secrets.get`
- **B.** Increase `spark.sql.shuffle.partitions` to 2000 and keep the credentials in the URL
- **C.** Call `repartition(64)` after the read and store the credentials in a notebook widget
- **D.** Convert the result to Pandas with `toPandas()` and write it with `to_sql`

Question 11

Objective: *Ingest semi-structured and unstructured data (for example, JSON and nested data) via Lakeflow Connect and other managed connectors into Unity Catalog-governed Delta tables.*

Clickstream events arrive as JSON with deeply nested fields that change often. Storing the JSON as a STRING makes queries slow, and a fixed struct schema breaks whenever the structure changes. The workspace runs Databricks Runtime 15.4 LTS or above.

Which approach does Databricks recommend for storing the payload?

- **A.** Keep a STRING column and parse it with `get_json_object` in every query
- **B.** Flatten every possible field into separate columns ahead of time
- **C.** Store the payload as `VARIANT` using `parse_json`, and query fields with path syntax such as `payload:device.os`
- **D.** Save each event as a separate file in a volume

Question 12

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A SQL-focused team wants a declarative, incremental ingestion table that picks up new Parquet files from cloud storage exactly once, without writing Python streaming code.

Which statement accomplishes this?

- **A.** `CREATE OR REFRESH STREAMING TABLE bronze_orders AS SELECT * FROM STREAM read_files('s3://landing/orders/', format => 'parquet')`
- **B.** `CREATE VIEW bronze_orders AS SELECT * FROM read_files('s3://landing/orders/')`
- **C.** `CREATE MATERIALIZED VIEW bronze_orders AS SELECT * FROM read_files('s3://landing/orders/')`
- **D.** `INSERT INTO bronze_orders SELECT * FROM read_files('s3://landing/orders/')` scheduled hourly

SECTION 3 Data Transformation and Modeling

Exam weight 22%

Question 13

Objective: *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

A silver job on Databricks Runtime 17.0 casts the bronze column `order_date` (STRING) to DATE. A few rows contain malformed strings such as `'2026-13-45'`, and the job now fails with a cast error. The requirement is to keep those rows with a NULL date.

Why does the job fail, and what is the correct fix?

- **A.** The column needs an explicit schema; add `.schema()` to the read
- **B.** ANSI mode is enabled by default in Runtime 17.0 and above, so invalid casts raise errors; use `try_cast(order_date AS DATE)`
- **C.** Photon does not support DATE types; disable Photon
- **D.** DATE casts need a time zone; set `spark.sql.session.timeZone`

Question 14

Objective: *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

In the bronze `inventory` DataFrame, `quantity` is sometimes null (meaning zero units) and `warehouse` is sometimes null (unknown). The silver table must replace null `quantity` with 0 and null `warehouse` with `'UNKNOWN'` without dropping any rows.

Which PySpark call does this?

- **A.** `df.dropna(subset=["quantity", "warehouse"])`
- **B.** `df.filter("quantity IS NOT NULL")`
- **C.** `df.na.drop(how="all")`
- **D.** `df.fillna({"quantity": 0, "warehouse": "UNKNOWN"})`

Question 15

Objective: *Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.*

Marketing wants the list of customers who have **never** placed an order. `customers` and `orders` share the column `customer_id`, and only the columns from `customers` are needed.

Which join produces this result most directly?

- **A.** `customers.join(orders, "customer_id", "inner")`
- **B.** `customers.join(orders, "customer_id", "left_anti")`
- **C.** `customers.join(orders, "customer_id", "left_semi")`
- **D.** `customers.crossJoin(orders)`

Question 16

Objective: *Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.*

Two regional DataFrames, `us_sales` and `eu_sales`, have the same column names but in a different order. They must be stacked into one DataFrame with every row kept, including exact duplicates, and each column must line up by name.

Which operation should the data engineer use?

- **A.** `us_sales.union(eu_sales)`
- **B.** `us_sales.union(eu_sales).distinct()`
- **C.** `us_sales.unionByName(eu_sales)`
- **D.** `us_sales.join(eu_sales, how="outer")`

Question 17

Objective: *Manipulate columns, rows, and table structures by adding, dropping, splitting, renaming column names, applying filters, and exploding arrays.*

The bronze `orders` table has an array column `items`. Some orders have an empty or null `items` array. The silver table needs one row per item, and orders without items must still appear, with a null item.

Which function should be used?

- **A.** `explode_outer("items")`
- **B.** `explode("items")`
- **C.** `flatten("items")`
- **D.** `array_distinct("items")`

Question 18

Objective: *Manipulate columns, rows, and table structures by adding, dropping, splitting, renaming column names, applying filters, and exploding arrays.*

The column `full_name` contains values like `'Park Minjun'`. The silver table must have separate `last_name` and `first_name` columns, and must not have the original `full_name` column.

Which PySpark code produces this result?

- **A.** `df.withColumnRenamed("full_name", "last_name")`
- **B.** `df.select(F.substring("full_name", 1, 3))`
- **C.**
```
df.withColumn("parts", F.split("full_name", " ")).withColumn("last_name",
  F.col("parts").getItem(0)).withColumn("first_name",
  F.col("parts").getItem(1)).drop("parts", "full_name")
```
- **D.** `df.explode("full_name")`

Question 19

Objective: *Perform data deduplication operations and aggregate operations on DataFrames, such as count, approximate count distinct, and mean, summary.*

A dashboard shows daily unique visitors across 20 billion events. The exact `countDistinct` takes 40 minutes. The business accepts a result within about 5% of the true value if it is much faster.

Which aggregation should the data engineer use?

- **A.** `count("visitor_id")`
- **B.** `approx_count_distinct("visitor_id")`
- **C.** `countDistinct("visitor_id")` with more shuffle partitions
- **D.** `mean("visitor_id")`

Question 20

Objective: *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

A notebook runs on serverless compute. An engineer tries to tune a slow aggregation by setting `spark.executor.memory` and `spark.sql.shuffle.partitions`, and then plans to re-run and compare runtimes.

Which statement is correct?

- **A.** Both properties can be set on serverless compute
- **B.** Neither property can be set on serverless compute
- **C.** Only `spark.executor.memory` can be set, because serverless controls partitioning
- **D.** Only `spark.sql.shuffle.partitions` can be set, because serverless does not support setting most Spark properties, including memory settings

---

Question 21

Objective: *Understand the difference between, and how to build, Gold layer objects such as materialized views, views, streaming tables, and tables for BI and analytics teams in Unity Catalog.*

An analytics team uses Databricks SQL and wants a Gold aggregate that is precomputed for dashboards and refreshed every hour without a separate job. It is created from a Pro SQL warehouse.

Which statement creates this object?

- **A.** `CREATE VIEW gold.daily_rev AS SELECT ...` together with a notebook cron
- **B.** `CREATE TEMP VIEW gold.daily_rev AS SELECT ...`
- **C.** `CREATE OR REFRESH STREAMING TABLE gold.daily_rev AS SELECT ... FROM silver.orders` with no STREAM keyword
- **D.** `CREATE MATERIALIZED VIEW gold.daily_rev SCHEDULE EVERY 1 HOUR AS SELECT ...`

Question 22

Objective: *Apply data quality checks and validation rules to ensure reliable Silver and Gold datasets.*

A Python Lakeflow Spark Declarative Pipeline defines a silver table. Three validation rules (non-null `id`, `amount > 0`, and a valid `country` code) are stored in one dictionary shared across datasets. Rows that break **any** rule must be dropped.

Which decorator should be applied?

A. `@dp.expect_all_or_drop(rules)`

B. `@dp.expect_all(rules)`

C. `@dp.expect_or_fail(rules)`

D. `@dp.expect_all_or_fail(rules)`

SECTION 4 Working with Lakeflow Jobs Exam weight 16%

Question 23

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

In a job DAG, `ingest_a` and `ingest_b` run in parallel. The downstream `build_report` task must run as long as at least one of them succeeds, so that partial data is still reported.

Which *Run if dependencies* setting should `build_report` use?

- **A.** All succeeded
- **B.** At least one succeeded
- **C.** All done
- **D.** None failed

Question 24

Objective: *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration*

A notebook task calls an external API that sometimes returns transient 503 errors. Re-running the task a few minutes later usually succeeds. The team wants this handled automatically without failing the whole job on the first error.

What should the data engineer configure?

- **A.** A retry policy on the task, with a number of retries and an interval between attempts
- **B.** An If/else condition task after every API call
- **C.** A For each task that runs the notebook three times
- **D.** A duration warning threshold on the task

Question 25

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

After a nightly pipeline updates the gold tables, stakeholders want the published AI/BI dashboard refreshed and a snapshot emailed to subscribers, all as part of the same job.

Which task type should be added downstream of the pipeline task?

- **A.** A notebook task that runs `display()` on the gold tables
- **B.** A SQL alert task
- **C.** A dashboard task using a serverless or pro SQL warehouse
- **D.** A Run Job task that points to the dashboard

Question 26

Objective: *Implement job schedules using Lakeflow Jobs with an understanding of trigger types (scheduled, file arrival, and table update)*

A finance job must run at 02:15 on weekdays in New York time. The team wants to see and edit the exact schedule expression.

How should the schedule be configured?

- **A.** A simple schedule set to every 24 hours
- **B.** An advanced scheduled trigger with the time zone America/New_York, using Quartz cron syntax
- **C.** A file arrival trigger on the finance landing folder
- **D.** A continuous trigger with a 24-hour pause parameter

Question 27

Objective: *Implement job schedules using Lakeflow Jobs with an understanding of trigger types (scheduled, file arrival, and table update)*

A partner drops new CSV files into a Unity Catalog volume at unpredictable times, sometimes several times a day. The processing job should start soon after each new file lands.

Which trigger type fits this requirement?

- **A.** A file arrival trigger on the volume path
- **B.** A table update trigger on the volume
- **C.** A scheduled trigger every 5 minutes
- **D.** A continuous trigger

Question 28

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

A job uses a pipeline task to run a Lakeflow Spark Declarative Pipeline. After a bug fix in a transformation, the team needs the next scheduled job run to recompute every table in the pipeline from scratch.

How can this be done from the job configuration?

- **A.** Delete the pipeline's checkpoint folder before the run
- **B.** Replace the pipeline task with a notebook task
- **C.** Enable the pipeline task's option to trigger a full refresh
- **D.** Set the task's retry count to 0

Question 29

Objective: *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration*

The task `count_rows` computes the number of records loaded. The downstream SQL task `log_load` needs that number as a parameter.

How should the value be passed between the tasks?

- **A.** Write it to a global temp view that `log_load` reads
- **B.** Store it in a notebook widget default
- **C.** Set an environment variable on the cluster
- **D.** Call `dbutils.jobs.taskValues.set(key="row_count", value=n)` in `count_rows` and reference `{{tasks.count_rows.values.row_count}}` in `log_load`

SECTION 5 Implementing CI/CD Exam weight 10%

Question 30

Objective: *Manage your code development workflow within the Databricks workspace UI, including creating and switching between branches in Databricks Git Folders (formerly Databricks Repos), committing and pushing changes, and creating pull requests using Databricks Git integration.*

A developer works in a Databricks Git folder connected to GitHub. They do not have permission to push to `main`. They need to change a notebook and get it reviewed and merged.

Which workflow is correct?

- **A.** Edit on `main` and click *Commit & Push*; Databricks opens the pull request automatically
- **B.** In the Git dialog, click *Create Branch*, make the change, click *Commit & Push*, then open the pull request in GitHub
- **C.** Export the notebook as a file and email it to the reviewer
- **D.** Clone the Git folder into the workspace user folder and edit it there

---

Question 31

Objective: *Deploy Declarative Automation Bundles (formerly Databricks Asset Bundles) to package, configure, and promote Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and other workspace assets across dev, test, and prod environments.*

A bundle's production target must run its jobs under a non-human identity so that runs do not break when employees leave and so that deployment and execution permissions are separate.

Which configuration should be used?

- **A.** `run_as` with `service_principal_name` under the prod target
- **B.** `run_as` with the team lead's `user_name` under the prod target
- **C.** `mode: development` on the prod target
- **D.** Leave `run_as` unset so jobs run as whoever deploys them

---

Question 32

Objective: *Understand the Databricks CLI to validate, deploy, and manage Declarative Automation Bundles (formerly Databricks Asset Bundles) and other workspace assets in automated CI/CD workflows.*

A feature branch deployed a bundle to the dev target. The feature has been abandoned, and every job, pipeline, and artifact that this deployment created should be removed.

Which command should the developer run?

- **A.** `databricks bundle deploy -t dev --force`
- **B.** `databricks bundle summary -t dev`
- **C.** `databricks workspace delete` on the notebook folder
- **D.** `databricks bundle destroy -t dev`

Question 33

Objective: *Deploy Declarative Automation Bundles (formerly Databricks Asset Bundles) to package, configure, and promote Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and other workspace assets across dev, test, and prod environments.*

A team is starting a new Python data project and wants a standard Declarative Automation Bundles layout, with `databricks.yml`, resources, and dev and prod targets, without writing the YAML by hand.

What should they run first?

A. `databricks bundle init` and choose a template such as `default-python`

B. `databricks bundle deploy -t prod`

C. `databricks bundle validate`

D. `databricks repos create`

SECTION 6 Troubleshooting, Monitoring, and Optimization

Exam weight 10%

Question 34

Objective: *Understand the features of Liquid Clustering and predictive optimization.*

A platform team maintains hundreds of Unity Catalog managed tables and currently runs custom nightly jobs for `OPTIMIZE`, `VACUUM`, and statistics collection. They want Databricks to decide automatically when each operation is worthwhile.

Which feature meets this requirement?

- **A.** Auto Loader file notification mode
- **B.** Delta change data feed
- **C.** Predictive optimization
- **D.** Adaptive query execution

Question 35

Objective: *Understand the features of Liquid Clustering and predictive optimization.*

An existing **unpartitioned** Delta table has two years of data. The team runs `ALTER TABLE events CLUSTER BY (customer_id)`, but queries on historical data show no improvement.

What explains this, and how can it be fixed?

- **A.** Liquid clustering applies only to new tables; recreate the table
- **B.** By default, clustering is not applied to previously written data; run `OPTIMIZE events FULL` to recluster the existing records
- **C.** `customer_id` must also be added as a partition column
- **D.** `VACUUM` must be run to activate clustering

Question 36

Objective: *Diagnose cluster startup failures, library conflicts, and out-of-memory issues.*

A classic cluster fails to start. Its event log shows the termination reason `AWS_INSUFFICIENT_INSTANCE_CAPACITY_FAILURE`.

Which action is most appropriate?

- **A.** Increase the driver memory in the Spark config
- **B.** Remove every library attached to the cluster
- **C.** Rewrite the notebook code to use fewer shuffles
- **D.** Choose a different availability zone or *auto*, switch instance types, or enable flexible node types

Question 37

Objective: *Diagnose cluster startup failures, library conflicts, and out-of-memory issues.*

Several notebooks share one classic cluster. One engineer needs `pandas==1.5.3` for a legacy script, but installing it as a cluster library breaks other teams' notebooks that expect the Runtime's newer version.

How should the engineer install the library to avoid this conflict?

- **A.** Install it as a notebook-scoped library with `%pip install pandas==1.5.3` and restart Python if needed
- **B.** Add it as a cluster library and ask the other teams to pin versions
- **C.** Put the install in a cluster-scoped init script
- **D.** Upgrade the Runtime to the newest version

Question 38

Objective: *Use the Lakeflow Jobs UI to monitor pipeline health by interpreting job statuses, viewing DAG-based task graphs to spot upstream blockers, and tracking pipeline run times and failure rates.*

In a job run's task graph, `extract` shows *Failed* and the downstream `transform` shows *Upstream failed*. `transform` uses the default dependency setting.

How should the data engineer interpret this?

- **A.** `transform` ran and failed because of a bug in its own code
- **B.** `transform` was skipped because a schedule overlap was detected
- **C.** `transform` did not run because its *Run if* condition (All succeeded) was not met, so troubleshooting should start at `extract`
- **D.** `transform` is still waiting for a retry and will run later automatically

---

SECTION 7 Governance and Security

Exam weight 15%

Question 39

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

The external table `main.sales.orders_ext` should become a Unity Catalog managed table so that it gets predictive optimization. Readers and writers should see minimal downtime, and the team wants the option to roll back.

Which command should the data engineer use?

- **A.** `ALTER TABLE main.sales.orders_ext SET MANAGED`
- **B.** `DROP TABLE` and then `CREATE TABLE` without a LOCATION clause
- **C.** `CREATE TABLE main.sales.orders_m DEEP CLONE main.sales.orders_ext` and point every job at the clone
- **D.** `ALTER TABLE main.sales.orders_ext SET LOCATION` to a managed path

Question 40

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

A data engineer runs `CREATE TABLE main.raw.logs (...) LOCATION 's3://acme-logs/raw/'` and gets a permission error, even though they have USE CATALOG, USE SCHEMA, and CREATE TABLE on the schema.

Which additional privilege is required?

- **A.** MODIFY on the schema
- **B.** `CREATE EXTERNAL TABLE` on the external location that covers the path
- **C.** SELECT on the catalog
- **D.** Workspace admin rights

Question 41

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

The group `bi_readers` must be able to read every current table in `main.gold` and every table added to that schema in the future, without additional grants each time.

Which grants achieve this with the least effort?

- **A.** `GRANT SELECT` on each table, repeated whenever a table is created
- **B.** `GRANT SELECT ON CATALOG main` only
- **C.** `GRANT USE CATALOG ON CATALOG main`, `GRANT USE SCHEMA ON SCHEMA main.gold`, and `GRANT SELECT ON SCHEMA main.gold`
- **D.** Make `bi_readers` the owner of `main.gold`

Question 42

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

A data steward must be able to manage permissions on the `main.finance` catalog and its objects, including granting access, without becoming the catalog owner.

Which privilege should be granted to the steward?

- **A.** `MANAGE` on the catalog
- **B.** `ALL PRIVILEGES` on the catalog
- **C.** `MODIFY` on the catalog
- **D.** `USE CATALOG` on the catalog

Question 43

Objective: *Understand column-level masking and row-level security to restrict data visibility based on user groups.*

In the table `hr.employees`, members of the group `hr_admins` must see the full `ssn` value, and everyone else must see `'***-**-****'`. The rule must apply to every query on the table, not only to queries through a view.

Which approach meets this requirement?

- **A.** Create a SQL UDF that returns `ssn` when `is_account_group_member('hr_admins')` and the masked value otherwise, then run `ALTER TABLE hr.employees ALTER COLUMN ssn SET MASK` with that function
- **B.** `REVOKE SELECT` on the table from everyone except `hr_admins`
- **C.** Create a second table without the `ssn` column
- **D.** Add a CHECK constraint on `ssn`

Question 44

Objective: *Understand column-level masking and row-level security to restrict data visibility based on user groups.*

In the table `sales.orders`, regional managers should see only rows where `region` matches their region group. Admins in the group `global_sales` should see all rows. The filter must apply directly on the table.

Which feature should be used?

- **A.** A column mask on `region`
- **B.** Separate tables per region with separate grants
- **C.** A row filter UDF applied with `ALTER TABLE sales.orders SET ROW FILTER region_filter ON (region)`
- **D.** A Delta CHECK constraint on `region`

Question 45

Objective: *Understand Unity Catalog ABAC policies to centrally control row-level filtering and column masking for sensitive data.*

Across many schemas in the catalog `prod`, every table tagged with the governed tag `region_scoped` must hide rows outside the user's region, and the `auditors` group must be exempt. Tables tagged in the future must be covered automatically.

Which approach is most suitable?

- **A.** Apply `SET ROW FILTER` to every tagged table individually
- **B.** Create a dynamic view for every tagged table
- **C.** Grant SELECT only on the schemas that contain untagged tables
- **D.** Create an ABAC row filter policy at the catalog level that targets the governed tag and lists `auditors` as exempt

ANSWERS

Answer Key

Answers and explanations

#	Answer	Section
1	**B**	D1
2	**C**	D1
3	**A**	D1
4	**D**	D2
5	**A**	D2
6	**C**	D2
7	**C**	D2
8	**B**	D2
9	**D**	D2
10	**A**	D2
11	**C**	D2
12	**A**	D2
13	**B**	D3
14	**D**	D3
15	**B**	D3
16	**C**	D3
17	**A**	D3
18	**C**	D3
19	**B**	D3
20	**D**	D3
21	**D**	D3
22	**A**	D3
23	**B**	D4

#	Answer	Section
24	**A**	D4
25	**C**	D4
26	**B**	D4
27	**A**	D4
28	**C**	D4
29	**D**	D4
30	**B**	D5
31	**A**	D5
32	**D**	D5
33	**A**	D5
34	**C**	D6
35	**B**	D6
36	**D**	D6
37	**A**	D6
38	**C**	D6
39	**A**	D7
40	**B**	D7
41	**C**	D7
42	**A**	D7
43	**A**	D7
44	**C**	D7
45	**D**	D7

REVIEW **Explanations** Rationale, distractors, and sources

Question 1 Section 1

Answer: B A Unity Catalog volume, accessed through `/Volumes/<catalog>/<schema>/<volume>/...`

Why: Volumes are Unity Catalog objects that govern non-tabular data such as images, PDFs, and audio files. Files are addressed with `/Volumes/<catalog>/<schema>/<volume>/<path>`, and that path works the same way from SQL, Python, and Spark.

Distractors: A: storing large files as table rows is an anti-pattern. C: views are defined over tables and cannot govern raw files. D: the DBFS root is a legacy location that Unity Catalog does not govern.

Source:
https://docs.databricks.com/aws/en/volumes/

Question 2 Section 1

Answer: C The file-based transaction log stored with the table's data files

Why: Delta Lake extends Parquet data files with a file-based transaction log that provides ACID transactions and scalable metadata handling. Every change is recorded as a commit in the log, and the log also makes time travel and versioning possible.

Distractors: A: footer statistics support data skipping, not transactions. B: Unity Catalog handles governance and naming, not per-write atomicity. D: driver memory is not persistent and is not shared between jobs.

Source:
https://docs.databricks.com/aws/en/delta/

Question 3 Section 1

Answer: A A serverless SQL warehouse

Why: Databricks recommends serverless SQL warehouses when they are available: compute is instant and elastic, management overhead is minimal, and automatic scaling reduces idle cost. Intelligent Workload Management watches the query queue and quickly provisions more clusters when wait times grow.

Distractors: B: a single cluster cannot scale out for concurrency. C: dedicated access mode is for one user, not hundreds of analysts. D: serverless jobs compute runs job workloads, not interactive BI queries.

Source:
https://docs.databricks.com/aws/en/compute/sql-warehouse/
https://docs.databricks.com/aws/en/compute/sql-warehouse/warehouse-behavior

Question 4 Section 2

Answer: D In the `_rescued_data` column that Auto Loader adds when it infers the schema

Why: When Auto Loader infers the schema, it automatically adds a rescued data column named `_rescued_data`. This column captures data that fails to parse because of type mismatches, case mismatches, or columns missing from the schema. You can rename it with the `rescuedDataColumn` option.

Distractors: A: values are never silently converted. B: type mismatches go to the rescued column; they do not fail the stream. C: no such default directory exists.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema

Question 5 Section 2

Answer: A `cloudFiles.cleanSource = MOVE` with `cloudFiles.cleanSource.moveDestination` set to a volume path

Why: `cloudFiles.cleanSource` (Runtime 16.4 LTS and above) automatically moves or deletes processed source files. `MOVE` archives them to `cloudFiles.cleanSource.moveDestination`, which can be a volume path. `cloudFiles.cleanSource.retentionDuration` controls how long Auto Loader waits before cleaning up.

Distractors: B: controls schema changes, not source files. C: affects how files are listed, not what happens to them after ingestion. D: VACUUM cleans the target table's old files, not the source folder.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/production
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/options

Question 6 Section 2

Answer: C `.trigger(availableNow=True)`

Why: `availableNow` processes all data that is available when the query starts, possibly in several batches, and then stops. This gives incremental batch behavior with checkpointed exactly-once tracking. `Trigger.Once` is deprecated in Runtime 11.3 LTS and above in favor of `availableNow`.

Distractors: A: the default trigger runs continuously and checks for new data every few milliseconds. B: keeps the cluster running and billing between batches. D: deprecated.

Source:
https://docs.databricks.com/aws/en/structured-streaming/triggers

Question 7 Section 2

Answer: C `FORMAT_OPTIONS ('mergeSchema' = 'true')` and `COPY_OPTIONS ('mergeSchema' = 'true')`

Why: To load into a schemaless placeholder table, set `mergeSchema` to `true` in both `FORMAT_OPTIONS` and `COPY_OPTIONS`. The empty table can be used only with COPY INTO until the first load; INSERT INTO and MERGE INTO are not supported on it.

Distractors: A: enables inference but does not let the table schema evolve. B: `force` turns off idempotency and has nothing to do with schema. D: placeholder tables are explicitly supported.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/copy-into/

Question 8 Section 2

Answer: B An ingestion gateway extracts change data and stages it in a Unity Catalog volume, and an ingestion pipeline applies the staged changes to Delta tables

Why: The gateway extracts change data from SQL Server, stages it in a Unity Catalog volume, and must run continuously so it captures changes before the source truncates its change logs. The ingestion pipeline runs on serverless compute and applies the staged data to Delta tables. SCD Type 2 requires CDC rather than Change Tracking.

Distractors: A: the connector reads changes incrementally, not full tables. C and D: these are not how the managed connector works.

Source:
https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/sql-server-pipeline

Question 9 Section 2

Answer: D Use the workspace UI to upload the file and create a Unity Catalog managed table from it

Why: The file upload UI lets users upload local files and create a Unity Catalog managed table from them without code. UI uploads have a 5 GB file size limit, so a 40 MB file is well within it.

Distractors: A and B are built for recurring pipelines and are too much for a one-time local file. C: external locations point to cloud storage, not local machines.

Source:
https://docs.databricks.com/aws/en/ingestion/file-upload/upload-data

Question 10 Section 2

Answer: A Set `partitionColumn="id"`, `lowerBound`, `upperBound`, and `numPartitions`, and read the credentials with `dbutils.secrets.get`

Why: Parallel JDBC reads need `partitionColumn`, `lowerBound`, `upperBound`, and `numPartitions` so that Spark issues concurrent range queries. The docs caution against setting `numPartitions` above about 50 because it can overload the source database. Credentials should come from secrets through `dbutils.secrets.get`, not from the URL.

Distractors: B and C: both change only what happens after the single-threaded read, so the read stays slow, and both leave credentials exposed. D: pulls all the data to the driver.

Source:
https://docs.databricks.com/aws/en/connect/external-systems/jdbc

Question 11 Section 2

Answer: C Store the payload as `VARIANT` using `parse_json`, and query fields with path syntax such as `payload:device.os`

Why: Databricks recommends `VARIANT` over JSON strings for semi-structured data because it performs better while keeping the schema flexible. Use `parse_json` to create values and `col:field.subfield` to query them. VARIANT requires Runtime 15.4 and above, and VARIANT columns cannot be used as clustering or partition keys.

Distractors: A: this is the slow pattern the scenario describes. B: breaks when the structure changes. D: not a queryable table design.

Source:
https://docs.databricks.com/aws/en/semi-structured/variant

Question 12 Section 2

Answer: A

```
CREATE OR REFRESH STREAMING TABLE bronze_orders AS SELECT * FROM STREAM
read_files('s3://landing/orders/', format => 'parquet')
```

Why: A streaming table defined with `STREAM read_files(...)` ingests files incrementally, and the docs say `read_files` uses Auto Loader when it runs in a streaming table query. Streaming tables are designed for append-only sources and process each input only once.

Distractors: B: a view re-reads every file on every query. C: a materialized view is for computed results, not exactly-once file ingestion. D: re-reads every file each run and creates duplicates.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/functions/read_files
https://docs.databricks.com/aws/en/ldp/streaming-tables

Question 13 Section 3

Answer: B ANSI mode is enabled by default in Runtime 17.0 and above, so invalid casts raise errors; use `try_cast(order_date AS DATE)`

Why: ANSI mode is enabled by default in Databricks Runtime 17.0 and above. Under ANSI mode, malformed casts throw runtime exceptions instead of returning NULL. `try_cast` behaves like CAST except that it returns NULL on a runtime error.

Distractors: A, C, and D do not address the ANSI behavior that causes the failure.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance

Question 14 Section 3

Answer: D `df.fillna({"quantity": 0, "warehouse": "UNKNOWN"})`

Why: `fillna` (also available as `na.fill`) accepts a dictionary of column-to-replacement values, so each column gets its own default and no rows are removed.

Distractors: A, B, and C all remove rows instead of filling values.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.fillna.html

Question 15 Section 3

Answer: B `customers.join(orders, "customer_id", "left_anti")`

Why: A left anti join returns the rows from the left side that have no match on the right side, and it returns only the left side's columns.

Distractors: A: returns customers who do have orders, repeated once per order. C: a left semi join returns customers who have at least one order. D: a Cartesian product.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.join.html

Question 16 Section 3

Answer: C `us_sales.unionByName(eu_sales)`

Why: `unionByName` matches columns by name rather than by position and keeps duplicates, which is the same as SQL UNION ALL. `allowMissingColumns=True` can fill columns that exist on only one side.

Distractors: A: `union` matches by position, so mismatched column orders produce wrong data. B: also removes duplicates. D: a join combines columns, not rows.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.unionByName.html

Question 17 Section 3

Answer: A `explode_outer("items")`

Why: `explode_outer` creates one row per array element and, unlike `explode`, produces a row with null when the array is null or empty, so those orders are kept.

Distractors: B: drops rows whose array is null or empty. C: merges nested arrays into one array and creates no rows. D: removes duplicate elements within the array.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.explode_outer.html

Question 18 Section 3

Answer: C `df.withColumn("parts", F.split("full_name", " ")).withColumn("last_name", F.col("parts").getItem(0)).withColumn("first_name", F.col("parts").getItem(1)).drop("parts", "full_name")`

Why: `split` turns the string into an array, `getItem(n)` takes individual elements, and `drop` removes the helper column and the original column.

Distractors: A: only renames the column. B: fixed-length substrings fail for names of different lengths. D: `explode` is a function for arrays, not a DataFrame method for strings.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.split.html

Question 19 Section 3

Answer: B `approx_count_distinct("visitor_id")`

Why: `approx_count_distinct` returns an estimated number of distinct values. By default its results are accurate within about 5%, which comes from its maximum relative standard deviation, and the `relativeSD` parameter can tighten that.

Distractors: A: counts all non-null values, not distinct ones. C: still exact and expensive. D: meaningless for IDs.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/functions/approx_count_distinct

Question 20 Section 3

Answer: D Only `spark.sql.shuffle.partitions` can be set, because serverless does not support setting most Spark properties, including memory settings

Why: Serverless compute supports only a short list of Spark properties, including `spark.sql.shuffle.partitions`, `spark.sql.session.timeZone`, `spark.sql.ansi.enabled`, and `spark.sql.files.maxPartitionBytes`. Memory settings are not on the list, so tuning on serverless means changing supported properties and re-measuring.

Distractors: A, B, and C contradict the documented list of supported properties.

Source:
https://docs.databricks.com/aws/en/spark/conf

Question 21 Section 3

Answer: D `CREATE MATERIALIZED VIEW gold.daily_rev SCHEDULE EVERY 1 HOUR AS SELECT ...`

Why: Materialized views support a `SCHEDULE EVERY n HOURS` clause (or CRON, or `TRIGGER ON UPDATE`). They can be created from a Pro or Serverless SQL warehouse or in a pipeline, and they are refreshed by serverless Lakeflow pipelines.

Distractors: A and B: views are not precomputed. C: a streaming table needs a streaming source and is meant for append-only ingestion.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-create-materialized-view

Question 22 Section 3

Answer: A `@dp.expect_all_or_drop(rules)`

Why: The `expect_all*` decorators take a dictionary of expectation names and constraints, so one set of rules can be reused. `expect_all_or_drop` drops any row that violates any of the constraints.

Distractors: B: warns and keeps invalid rows. C: applies to a single expectation, not a dictionary, and fails the update. D: fails the update instead of dropping rows.

Source:
https://docs.databricks.com/aws/en/ldp/expectations

Question 23 Section 4

Answer: B At least one succeeded

Why: *At least one succeeded* runs the task when any dependency succeeds. If none succeed, the task is marked Upstream failed.

Distractors: A: the default, which requires every dependency to succeed. C: runs even when both fail. D: requires that no dependency failed.

Source:
https://docs.databricks.com/aws/en/jobs/run-if

Question 24 Section 4

Answer: A A retry policy on the task, with a number of retries and an interval between attempts

Why: Each task can have a retry policy that sets how many times, and at what interval, a failed task run is retried. This is the built-in control flow for transient failures.

Distractors: B: branches on values and does not retry. C: always runs every iteration, even when the first one succeeds. D: warns about duration and does not retry.

Source:
https://docs.databricks.com/aws/en/jobs/configure-task

Question 25 Section 4

Answer: C A dashboard task using a serverless or pro SQL warehouse

Why: A dashboard task refreshes a published dashboard as part of a job and can email subscribers when it refreshes. It requires a serverless or pro SQL warehouse, or it uses the dashboard's default warehouse.

Distractors: A: `display()` does not refresh the dashboard. B: alerts evaluate conditions and do not refresh dashboards. D: Run Job triggers another job, not a dashboard.

Source:
https://docs.databricks.com/aws/en/jobs/dashboard

Question 26 Section 4

Answer: B An advanced scheduled trigger with the time zone America/New_York, using Quartz cron syntax

Why: An advanced schedule lets you set the period, start time, and time zone, and *Show Cron Syntax* displays the schedule in Quartz cron syntax for editing. A simple schedule cannot set the time of the first run.

Distractors: A: cannot target 02:15 on weekdays only. C: data-driven, not time-based. D: not a valid pattern.

Source:
https://docs.databricks.com/aws/en/jobs/scheduled

Question 27 Section 4

Answer: A A file arrival trigger on the volume path

Why: File arrival triggers watch external locations or Unity Catalog volumes, including their subdirectories, and start a run when new files appear. Overwriting an existing file with a file of the same name does not start a run.

Distractors: B: table update triggers watch tables, not volumes. C: polls whether or not files arrive. D: keeps compute running all the time.

Source:
https://docs.databricks.com/aws/en/jobs/file-arrival-triggers

Question 28 Section 4

Answer: C Enable the pipeline task's option to trigger a full refresh

Why: When you configure a pipeline task in the Jobs UI, you can choose to trigger a full refresh of the pipeline.

Distractors: A: manual checkpoint manipulation is not supported practice. B: loses the pipeline semantics. D: retries have no effect on refresh mode.

Source:
https://docs.databricks.com/aws/en/jobs/pipeline

Question 29 Section 4

Answer: D Call `dbutils.jobs.taskValues.set(key="row_count", value=n)` in `count_rows` and reference `{{tasks.count_rows.values.row_count}}` in `log_load`

Why: Task values let upstream tasks publish JSON-serializable values of up to 48 KiB. Downstream tasks reference them with `{{tasks.<task_name>.values.<key>}}`, which the docs recommend over calling `dbutils.jobs.taskValues.get()`.

Distractors: A: temp views do not survive across tasks and compute. B: widget defaults are static. C: not a supported way to pass values between tasks.

Source:
https://docs.databricks.com/aws/en/jobs/task-values

Question 30 Section 5

Answer: B In the Git dialog, click *Create Branch*, make the change, click *Commit & Push*, then open the pull request in GitHub

Why: Branches are created from the Git dialog with *Create Branch*, and changes are pushed with *Commit & Push*. When you cannot commit to the default branch, the docs say to create a new branch and use the Git provider's interface to create a pull request and merge it.

Distractors: A: pushing to `main` is not allowed here, and pull requests are opened in the Git provider. C and D: skip Git entirely.

Source:
https://docs.databricks.com/aws/en/repos/git-operations-with-repos

Question 31 Section 5

Answer: A `run_as` with `service_principal_name` under the prod target

Why: `run_as` can be set at the top level, per target, or per resource, to a `user_name` or a `service_principal_name`. The docs describe a service principal as the most secure identity for production targets.

Distractors: B: tied to one person. C: development mode prefixes names and pauses schedules and is not meant for prod. D: runs with whichever human deployed.

Source:
https://docs.databricks.com/aws/en/dev-tools/bundles/run-as

Question 32 Section 5

Answer: D `databricks bundle destroy -t dev`

Why: `bundle destroy` deletes the jobs, pipelines, other resources, and artifacts that were previously deployed for the target. The action is permanent.

Distractors: A: redeploys the bundle. B: only shows the deployed resources and their links. C: removes files only, and the jobs and pipelines remain.

Source:
https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands

Question 33 Section 5

Answer: A `databricks bundle init` and choose a template such as `default-python`

Why: `bundle init` creates a new bundle from a template. When run without arguments it lists the default templates, such as `default-python`. Validate and deploy come after the bundle exists.

Distractors: B and C: need an existing bundle. D: creates a Git folder, not a bundle.

Source:
https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands

Question 34 Section 6

Answer: C Predictive optimization

Why: Predictive optimization automatically runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` on Unity Catalog managed tables, and does not apply to external tables. It runs on serverless jobs compute and is enabled by default for accounts created on or after November 11, 2024.

Distractors: A: ingestion. B: tracks row-level changes. D: optimizes query plans at runtime, not table maintenance.

Source:
https://docs.databricks.com/aws/en/optimizations/predictive-optimization

Question 35 Section 6

Answer: B By default, clustering is not applied to previously written data; run `OPTIMIZE events FULL` to recluster the existing records

Why: Enabling clustering on an existing unpartitioned table with `ALTER TABLE ... CLUSTER BY` does not recluster data that was already written. `OPTIMIZE ... FULL`, optionally with a WHERE clause, forces reclustering of existing data.

Distractors: A: existing tables can enable clustering. C: liquid clustering cannot be combined with partitioning. D: VACUUM removes old files and does not cluster data.

Source:
https://docs.databricks.com/aws/en/delta/clustering

Question 36 Section 6

Answer: D Choose a different availability zone or *auto*, switch instance types, or enable flexible node types

Why: This code means AWS does not have enough capacity for the requested instance type in that availability zone. The documented fixes are to try another zone or *auto*, switch instance types, or enable flexible node types so that Databricks can fall back to compatible instances.

Distractors: A, B, and C address code or configuration, not a cloud capacity problem. Use the cluster event log's error code to find the documented fix.

Source:
https://docs.databricks.com/aws/en/compute/troubleshooting/cluster-error-codes

Question 37 Section 6

Answer: A Install it as a notebook-scoped library with `%pip install pandas==1.5.3` and restart Python if needed

Why: Notebook-scoped libraries installed with `%pip` are available only to the current notebook and its jobs, and other notebooks on the same cluster are not affected. `%pip` does not restart Python automatically, so `dbutils.library.restartPython()` may be needed.

Distractors: B and C: cluster-wide changes that affect everyone on the cluster. D: does not isolate the legacy version.

Source:
https://docs.databricks.com/aws/en/libraries/notebooks-python-libraries

Question 38 Section 6

Answer: C `transform` did not run because its *Run if* condition (All succeeded) was not met, so troubleshooting should start at `extract`

Why: With the default *All succeeded* setting, a task whose dependencies did not all succeed is marked *Upstream failed* and does not run. The root cause is the upstream task, so the task graph shows exactly where to look.

Distractors: A: the task never ran. B and D: this status does not mean either of these.

Source:
https://docs.databricks.com/aws/en/jobs/run-if
https://docs.databricks.com/aws/en/jobs/monitor

Question 39 Section 7

Answer: A `ALTER TABLE main.sales.orders_ext SET MANAGED`

Why: `SET MANAGED` copies the data and Delta log to managed storage while readers and writers keep working, then switches over with brief write downtime. It can be rolled back within 14 days with `UNSET MANAGED`.

Distractors: B: loses the data and history and causes downtime. C: creates a new table, so every consumer must switch and the history is lost. D: not the documented conversion method.

Source:
https://docs.databricks.com/aws/en/tables/convert-to-managed

Question 40 Section 7

Answer: B `CREATE EXTERNAL TABLE` on the external location that covers the path

Why: A table created with a LOCATION clause is an external table. Creating one requires `CREATE EXTERNAL TABLE` on an external location, in addition to USE CATALOG, USE SCHEMA, and CREATE TABLE.

Distractors: A and C: do not grant access to the storage path. D: not required, and far too broad.

Source:
https://docs.databricks.com/aws/en/tables/external

Question 41 Section 7

Answer: C `GRANT USE CATALOG ON CATALOG main`, `GRANT USE SCHEMA ON SCHEMA main.gold`, and `GRANT SELECT ON SCHEMA main.gold`

Why: Privileges granted on a schema apply to the current and future objects in it. Reading also requires USE CATALOG and USE SCHEMA on the parent objects.

Distractors: A: manual and repeated. B: too broad, and still missing USE CATALOG and USE SCHEMA. D: ownership gives far more than read access.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

Question 42 Section 7

Answer: A `MANAGE` on the catalog

Why: `MANAGE` lets a principal manage privileges on, transfer ownership of, and delete an object without being its owner. On a catalog it also applies to child schemas and tables.

Distractors: B: grants data access privileges, not privilege management. C: allows writing data. D: only lets the principal use the catalog.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

Question 43 Section 7

Answer: A Create a SQL UDF that returns `ssn` when `is_account_group_member('hr_admins')` and the masked value otherwise, then run `ALTER TABLE hr.employees ALTER COLUMN ssn SET MASK` with that function

Why: A column mask is a SQL UDF that takes the column value and returns either the original or a masked version, and it is attached to the table with `ALTER COLUMN ... SET MASK`. Because it is attached to the table itself, it applies to every query.

Distractors: B: blocks the whole table. C: duplicates data and adds governance overhead. D: validates writes and does not mask reads.

Source:
https://docs.databricks.com/aws/en/tables/row-and-column-filters

Question 44 Section 7

Answer: C A row filter UDF applied with `ALTER TABLE sales.orders SET ROW FILTER region_filter ON (region)`

Why: A row filter is a SQL UDF that is evaluated for each row at query time and returns a boolean. It is applied with `SET ROW FILTER <fn> ON (<cols>)`, and the function can use group membership checks such as `is_account_group_member()`.

Distractors: A: masks values instead of hiding rows. B: duplicates the data. D: constrains writes, not reads.

Source:
https://docs.databricks.com/aws/en/tables/row-and-column-filters

Question 45 Section 7

Answer: D Create an ABAC row filter policy at the catalog level that targets the governed tag and lists `auditors` as exempt

Why: ABAC row filter policies use a UDF to exclude rows, target objects through governed tags, and can be attached at the catalog or schema level. Exempt principals see unfiltered data. One policy therefore covers current and future tagged tables.

Distractors: A and B: configured table by table, so new tables are missed. C: does not filter rows.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/policies
https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/

WARNING Q14-Q18 (PySpark DataFrame API behavior such as `fillna`, `left_anti`, `unionByName`, `explode_outer`, `split`) are sourced from the Apache Spark API reference, because docs.databricks.com does not document these functions individually. All other answers are grounded in docs.databricks.com, checked Sep 29, 2026.